
5

OBJECT-ORIENTED PROGRAMMING



As we will see, the basis of a good architecture is the understanding and application of the principles of object-oriented design (OO). But just what is OO?

One answer to this question is “The combination of data and function.” Although often cited, this is a very unsatisfying answer because it implies that $o.f()$ is somehow different from $f(o)$. This is absurd. Programmers were passing data structures into functions long before 1966, when Dahl and Nygaard moved the function call stack frame to the heap and invented OO.

Another common answer to this question is “A way to model the real world.” This is an evasive answer at best. What does “modeling the real world” actually mean, and why is it something we would want to do? Perhaps this statement is intended to imply that OO makes software easier to understand because it has a closer

relationship to the real world—but even that statement is evasive and too loosely defined. It does not tell us what OO is.

Some folks fall back on three magic words to explain the nature of OO: *encapsulation*, *inheritance*, and *polymorphism*. The implication is that OO is the proper admixture of these three things, or at least that an OO language must support these three things.

Let's examine each of these concepts in turn.

ENCAPSULATION?

The reason encapsulation is cited as part of the definition of OO is that OO languages provide easy and effective encapsulation of data and function. As a result, a line can be drawn around a cohesive set of data and functions. Outside of that line, the data is hidden and only some of the functions are known. We see this concept in action as the private data members and the public member functions of a class.

This idea is certainly not unique to OO. Indeed, we had perfect encapsulation in C. Consider this simple C program:

[Click here to view code image](#)

point.h

```
struct Point;
struct Point* makePoint(double x, double y);
double distance (struct Point *p1, struct Point *p2);
```

[Click here to view code image](#)

point.c

```
#include "point.h"
#include <stdlib.h>
#include <math.h>

struct Point {
    double x,y;
};

struct Point* makepoint(double x, double y) {
```

```

    struct Point* p = malloc(sizeof(struct Point));
    p->x = x;
    p->y = y;
    return p;
}

double distance(struct Point* p1, struct Point* p2) {
    double dx = p1->x - p2->x;
    double dy = p1->y - p2->y;
    return sqrt(dx*dx+dy*dy);
}

```

The users of `point.h` have no access whatsoever to the members of `struct Point`. They can call the `makePoint()` function, and the `distance()` function, but they have absolutely no knowledge of the implementation of either the `Point` data structure or the functions.

This is perfect encapsulation—in a non-OO language. C programmers used to do this kind of thing all the time. We would forward declare data structures and functions in header files, and then implement them in implementation files. Our users never had access to the elements in those implementation files.

But then came OO in the form of C++—and the perfect encapsulation of C was broken.

The C++ compiler, for technical reasons,¹ needed the member variables of a class to be declared in the header file of that class. So our `Point` program changed to look like this:

[Click here to view code image](#)

`point.h`

```

class Point {
public:
    Point(double x, double y);
    double distance(const Point& p) const;

private:
    double x;
    double y;
};

```

[Click here to view code image](#)

point.cc

```
#include "point.h"
#include <math.h>

Point::Point(double x, double y)
: x(x), y(y)
{}

double Point::distance(const Point& p) const {
    double dx = x-p.x;
    double dy = y-p.y;
    return sqrt(dx*dx + dy*dy);
}
```

Clients of the header file `point.h` know about the member variables `x` and `y`! The compiler will prevent access to them, but the client still knows they exist. For example, if those member names are changed, the `point.cc` file must be recompiled! Encapsulation has been broken.

Indeed, the way encapsulation is partially repaired is by introducing the `public`, `private`, and `protected` keywords into the language. This, however, was a *hack* necessitated by the technical need for the compiler to see those variables in the header file.

Java and C# simply abolished the header/implementation split altogether, thereby weakening encapsulation even more. In these languages, it is impossible to separate the declaration and definition of a class.

For these reasons, it is difficult to accept that OO depends on strong encapsulation. Indeed, many OO languages² have little or no enforced encapsulation.

OO certainly does depend on the idea that programmers are well-behaved enough to not circumvent encapsulated data. Even so, the languages that claim to provide OO have only weakened the once perfect encapsulation we enjoyed with C.

INHERITANCE?

If OO languages did not give us better encapsulation, then they certainly gave us inheritance.

Well—sort of. Inheritance is simply the redeclaration of a group of variables and functions within an enclosing scope. This is something C programmers³ were able to do manually long before there was an OO language.

Consider this addition to our original `point.h` C program:

[Click here to view code image](#)

`namedPoint.h`

```
struct NamedPoint;

struct NamedPoint* makeNamedPoint(double x, double y, char* name);
void setName(struct NamedPoint* np, char* name);
char* getName(struct NamedPoint* np);
```

[Click here to view code image](#)

`namedPoint.c`

```
#include "namedPoint.h"
#include <stdlib.h>

struct NamedPoint {
    double x,y;
    char* name;
};

struct NamedPoint* makeNamedPoint(double x, double y, char* name)
{
    struct NamedPoint* p = malloc(sizeof(struct NamedPoint));
    p->x = x;
    p->y = y;
    p->name = name;
    return p;
}

void setName(struct NamedPoint* np, char* name) {
    np->name = name;
}

char* getName(struct NamedPoint* np) {
    return np->name;
}
```

[Click here to view code image](#)

main.c

```
#include "point.h"
#include "namedPoint.h"
#include <stdio.h>

int main(int ac, char** av) {
    struct NamedPoint* origin = makeNamedPoint(0.0, 0.0, "origin");
    struct NamedPoint* upperRight = makeNamedPoint (1.0, 1.0,
"upperRight");
    printf("distance=%f\n",
        distance(
            (struct Point*) origin,
            (struct Point*) upperRight));
}
```

If you look carefully at the `main` program, you'll see that the `NamedPoint` data structure acts as though it is a derivative of the `Point` data structure. This is because the order of the first two fields in `NamedPoint` is the same as `Point`. In short, `NamedPoint` can masquerade as `Point` because `NamedPoint` is a pure superset of `Point` and maintains the ordering of the members that correspond to `Point`.

This kind of trickery was a common practice⁴ of programmers prior to the advent of OO. In fact, such trickery is how C++ implements single inheritance.

Thus we might say that we had a kind of inheritance long before OO languages were invented. That statement wouldn't quite be true, though. We had a trick, but it's not nearly as convenient as true inheritance. Moreover, multiple inheritance is a considerably more difficult to achieve by such trickery.

Note also that in `main.c`, I was forced to cast the `NamedPoint` arguments to `Point`. In a real OO language, such upcasting would be implicit.

It's fair to say that while OO languages did not give us something completely brand new, it did make the masquerading of data structures significantly more convenient.

To recap: We can award no point to OO for encapsulation, and perhaps a half-point for inheritance. So far, that's not such a great score.

But there's one more attribute to consider.

POLYMORPHISM?

Did we have polymorphic behavior before OO languages? Of course we did. Consider this simple C `copy` program.

[Click here to view code image](#)

```
#include <stdio.h>

void copy() {
    int c;
    while ((c=getchar()) != EOF)
        putchar(c);
}
```

The function `getchar()` reads from `STDIN`. But which device is `STDIN`? The `putchar()` function writes to `STDOUT`. But which device is that? These functions are *polymorphic*—their behavior depends on the type of `STDIN` and `STDOUT`.

It's as though `STDIN` and `STDOUT` are Java-style interfaces that have implementations for each device. Of course, there are no interfaces in the example C program—so how does the call to `getchar()` actually get delivered to the device driver that reads the character?

The answer to that question is pretty straightforward. The UNIX operating system requires that every IO device driver provide five standard functions:⁵ `open`, `close`, `read`, `write`, and `seek`. The signatures of those functions must be identical for every IO driver.

The `FILE` data structure contains five pointers to functions. In our example, it might look like this:

[Click here to view code image](#)

```
struct FILE {
    void (*open)(char* name, int mode);
    void (*close)();
    int (*read)();
    void (*write)(char);
    void (*seek)(long index, int mode);
};
```

The IO driver for the console will define those functions and load up a `FILE` data structure with their addresses—something like this:

[Click here to view code image](#)

```
#include "file.h"

void open(char* name, int mode) { /*...*/ }
void close() { /*...*/ };
int read() { int c; /*...*/ return c; }
void write(char c) { /*...*/ }
void seek(long index, int mode) { /*...*/ }

struct FILE console = {open, close, read, write, seek};
```

Now if `STDIN` is defined as a `FILE*`, and if it points to the console data structure, then `getchar()` might be implemented this way:

[Click here to view code image](#)

```
extern struct FILE* STDIN;

int getchar() {
    return STDIN->read();
}
```

In other words, `getchar()` simply calls the function pointed to by the `read` pointer of the `FILE` data structure pointed to by `STDIN`.

This simple trick is the basis for all polymorphism in OO. In C++, for example, every virtual function within a class has a pointer in a table called a `vtable`, and all calls to virtual functions go through that table. Constructors of derivatives simply load their versions of those functions into the `vtable` of the object being created.

The bottom line is that polymorphism is an application of pointers to functions. Programmers have been using pointers to functions to achieve polymorphic behavior since Von Neumann architectures were first implemented in the late 1940s. In other words, OO has provided nothing new.

Ah, but that's not quite correct. OO languages may not have given us polymorphism, but they have made it much safer and much more convenient.

The problem with explicitly using pointers to functions to create polymorphic

behavior is that pointers to functions are *dangerous*. Such use is driven by a set of manual conventions. You have to remember to follow the convention to initialize those pointers. You have to remember to follow the convention to call all your functions through those pointers. If any programmer fails to remember these conventions, the resulting bug can be devilishly hard to track down and eliminate.

OO languages eliminate these conventions and, therefore, these dangers. Using an OO language makes polymorphism trivial. That fact provides an enormous power that old C programmers could only dream of. On this basis, we can conclude that OO imposes discipline on indirect transfer of control.

THE POWER OF POLYMORPHISM

What's so great about polymorphism? To better appreciate its charms, let's reconsider the example `copy` program. What happens to that program if a new IO device is created? Suppose we want to use the `copy` program to copy data from a handwriting recognition device to a speech synthesizer device: How do we need to change the `copy` program to get it to work with those new devices?

We don't need any changes at all! Indeed, we don't even need to recompile the `copy` program. Why? Because the source code of the `copy` program does not depend on the source code of the IO drivers. As long as those IO drivers implement the five standard functions defined by `FILE`, the `copy` program will be happy to use them.

In short, the IO devices have become plugins to the `copy` program.

Why did the UNIX operating system make IO devices plugins? Because we learned, in the late 1950s, that our programs should be *device independent*. Why? Because we wrote lots of programs that were *device dependent*, only to discover that we really wanted those programs to do the same job but use a different device.

For example, we often wrote programs that read input data from decks of cards,⁶ and then punched new decks of cards as output. Later, our customers stopped giving us decks of cards and started giving us reels of magnetic tape. This was very inconvenient, because it meant rewriting large portions of the original program. It would be very convenient if the same program worked interchangeably with cards or tape.

The plugin architecture was invented to support this kind of IO device independence, and has been implemented in almost every operating system since its introduction.

Even so, most programmers did not extend the idea to their own programs, because using pointers to functions was dangerous.

OO allows the plugin architecture to be used anywhere, for anything.

DEPENDENCY INVERSION

Imagine what software was like before a safe and convenient mechanism for polymorphism was available. In the typical calling tree, main functions called high-level functions, which called mid-level functions, which called low-level functions. In that calling tree, however, source code dependencies inexorably followed the flow of control ([Figure 5.1](#)).

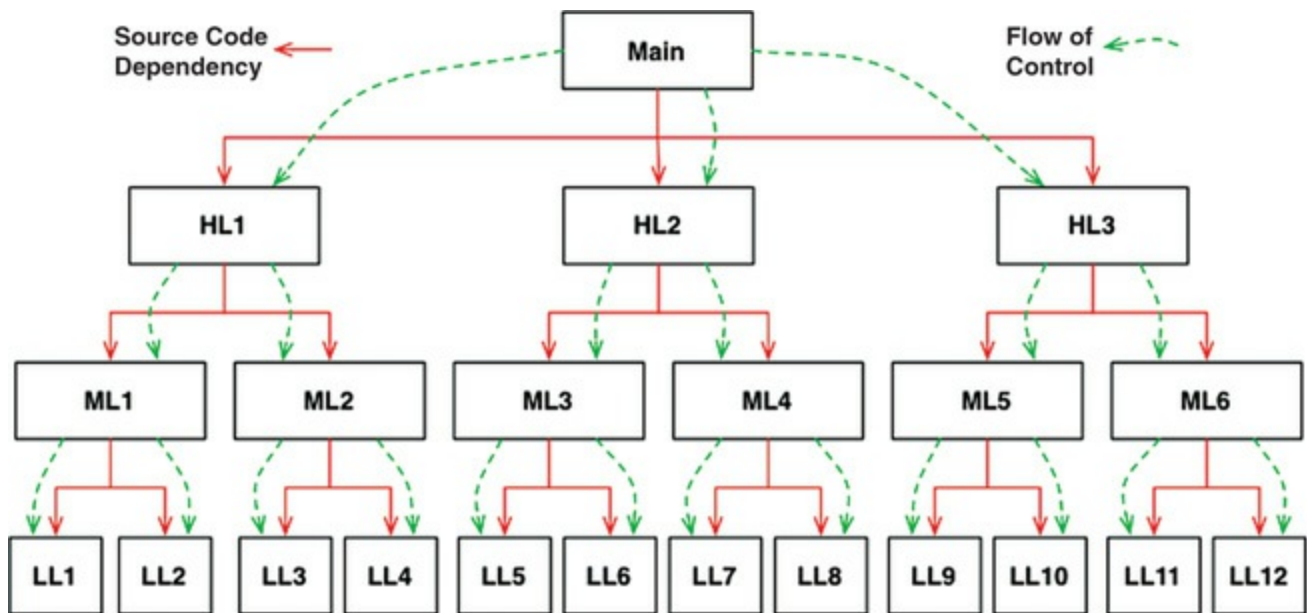


Figure 5.1 Source code dependencies versus flow of control

For `main` to call one of the high-level functions, it had to mention the name of the module that contained that function. In C, this was a `#include`. In Java, it was an `import` statement. In C#, it was a `using` statement. Indeed, every caller was forced to mention the name of the module that contained the callee.

This requirement presented the software architect with few, if any, options. The flow of control was dictated by the behavior of the system, and the source code dependencies were dictated by that flow of control.

When polymorphism is brought into play, however, something very different can happen ([Figure 5.2](#)).

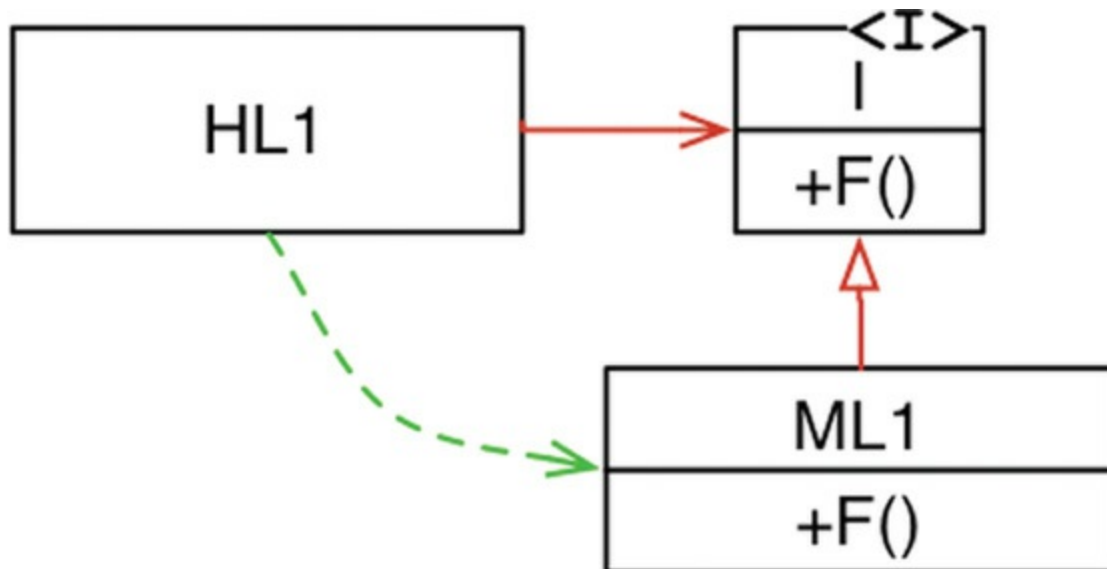


Figure 5.2 Dependency inversion

In [Figure 5.2](#), module `HL1` calls the `F()` function in module `ML1`. The fact that it calls this function through an interface is a source code contrivance. At runtime, the interface doesn't exist. `HL1` simply calls `F()` within `ML1`.⁷

Note, however, that the source code dependency (the inheritance relationship) between `ML1` and the interface `I` points in the opposite direction compared to the flow of control. This is called *dependency inversion*, and its implications for the software architect are profound.

The fact that OO languages provide safe and convenient polymorphism means that *any source code dependency, no matter where it is, can be inverted*.

Now look back at that calling tree in [Figure 5.1](#), and its many source code dependencies. Any of those source code dependencies can be turned around by inserting an interface between them.

With this approach, software architects working in systems written in OO languages have *absolute control* over the direction of all source code dependencies in the system. They are not constrained to align those dependencies with the flow of control. No matter which module does the calling and which module is called, the software architect can point the source code dependency in either direction.

That is power! That is the power that OO provides. That's what OO is really all about—at least from the architect's point of view.

What can you do with that power? As an example, you can rearrange the source code dependencies of your system so that the database and the user interface (UI) depend on the business rules ([Figure 5.3](#)), rather than the other way around.

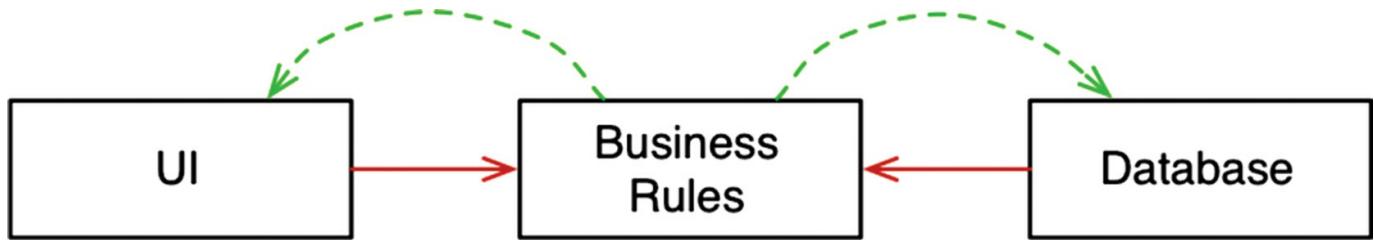


Figure 5.3 The database and the user interface depend on the business rules

This means that the UI and the database can be plugins to the business rules. It means that the source code of the business rules never mentions the UI or the database.

As a consequence, the business rules, the UI, and the database can be compiled into three separate components or deployment units (e.g., jar files, DLLs, or Gem files) that have the same dependencies as the source code. The component containing the business rules will not depend on the components containing the UI and database.

In turn, the business rules can be *deployed independently* of the UI and the database. Changes to the UI or the database need not have any effect on the business rules. Those components can be deployed separately and independently.

In short, when the source code in a component changes, only that component needs to be redeployed. This is *independent deployability*.

If the modules in your system can be deployed independently, then they can be developed independently by different teams. That's *independent developability*.

CONCLUSION

What is OO? There are many opinions and many answers to this question. To the software architect, however, the answer is clear: OO is the ability, through the use of polymorphism, to gain absolute control over every source code dependency in the system. It allows the architect to create a plugin architecture, in which modules that contain high-level policies are independent of modules that contain low-level details. The low-level details are relegated to plugin modules that can be deployed and developed independently from the modules that contain high-level policies.

1. The C++ compiler needs to know the size of the instances of each class.
2. For example, Smalltalk, Python, JavaScript, Lua, and Ruby.
3. Not just C programmers: Most languages of that era had the capability to masquerade one data structure as another.
4. Indeed it still is.
5. UNIX systems vary; this is just an example.
6. Punched cards—IBM Hollerith cards, 80 columns wide. I'm sure many of you have never even seen one of these, but they were commonplace in the 1950s, 1960s, and even 1970s.
7. Albeit indirectly.